

Sprint 2 Notes: JSX, Components, Fragments, and Module Exports

Sprint 2 Goal

By the end of Sprint 2, you should be able to build a React UI using small reusable components instead of putting everything inside `App.jsx`.

You should be able to:

- Write JSX correctly.
- Create React components.
- Use JavaScript expressions inside JSX.
- Use `className` instead of `class`.
- Name components correctly.
- Return multiple JSX elements using a parent element or a fragment.
- Split components into separate files.
- Use default exports.
- Use named exports.
- Import components correctly.
- Start organizing a beginner-friendly React project structure.

Simple version:

Sprint 2 teaches you how to break a React screen into clean, reusable pieces.

1. Current Position After Sprint 1

In Sprint 1, you learned the basic orientation:

- React is a JavaScript library for building user interfaces.
- React apps are made from reusable pieces called components.
- An SPA loads one main HTML page and changes the screen using JavaScript.
- Vite helps you create and run a React project quickly.
- `src/App.jsx` is usually the first file you edit in a new Vite React project.

Sprint 2 continues from there.

In Sprint 1, you may have written most of your UI directly inside `App.jsx`.

In Sprint 2, you learn how to split the UI into smaller files and components.

2. What Is JSX?

JSX stands for JavaScript XML.

You do not need to memorize the full name. The important idea is this:

JSX lets you write UI code that looks like HTML inside JavaScript.

Example:

```
export default function App() {  
  return <h1>Hello React</h1>;  
}
```

This looks like HTML:

```
<h1>Hello React</h1>
```

But inside a React file, it is JSX.

React uses JSX to describe what should appear on the screen.

2.1 JSX Is Not Exactly HTML

JSX looks similar to HTML, but it has different rules because it is written inside JavaScript.

Important JSX differences:

- Use `className`, not `class`.
 - JavaScript expressions go inside `{}`.
 - Every tag must be closed.
 - Components must start with a capital letter.
 - A component must return one parent value.
-

3. JSX Rule 1: Use `className`, Not `class`

In normal HTML, you write:

```
<div class="card">Hello</div>
```

In React JSX, write:

```
<div className="card">Hello</div>
```

Why?

Because `class` is a reserved word in JavaScript. Since JSX is JavaScript syntax, React uses `className` instead.

Correct:

```
function Card() {  
  return <div className="card">This is a card</div>;  
}
```

Incorrect:

```
function Card() {  
  return <div class="card">This is a card</div>;  
}
```

Beginner rule:

In React, whenever you want to add a CSS class, write `className`.

4. JSX Rule 2: Put JavaScript Expressions Inside `{ }`

JSX can show JavaScript values.

To place JavaScript inside JSX, use curly braces.

Example:

```
function Greeting() {  
  const name = "Sangeeth";
```

```
    return <h1>Hello {name}</h1>;  
  }
```

Output:

Hello Sangeeth

The value of the `name` variable is inserted into the JSX.

4.1 JSX Can Use Calculations Inside `{}`

Example:

```
function Total() {  
  const price = 100;  
  const quantity = 3;  
  
  return <p>Total: {price * quantity}</p>;  
}
```

Output:

Total: 300

React calculates `price * quantity` and displays the result.

4.2 JSX Can Use Conditional Expressions Inside `{}`

Example:

```
function AgeMessage() {  
  const age = 20;  
  
  return <p>{age >= 18 ? "Adult" : "Minor"}</p>;  
}
```

This uses a ternary expression.

If `age` is greater than or equal to `18`, it shows:

Adult

Otherwise, it shows:

Minor

4.3 Expressions vs Statements

Inside JSX curly braces, you normally write JavaScript expressions.

An expression produces a value.

Good:

```
<p>{age >= 18 ? "Adult" : "Minor"}</p>
```

Bad:

```
<p>{if (age >= 18) "Adult"}</p>
```

Why is the second one wrong?

Because `if` is a statement, not an expression.

Beginner rule:

Inside JSX `{ }`, use values, variables, calculations, function calls, or ternary expressions. Do not place a full `if` statement directly inside JSX.

5. JSX Rule 3: Every Tag Must Close

In HTML, some tags can sometimes be written without a closing slash.

Example:

```

```

In JSX, this must be self-closing:

```

```

Same for inputs:

```
<input type="text" />
```

Same for custom React components:

```
<Header />
```

Beginner rule:

In JSX, close every tag.

6. What Is a React Component?

A React component is a JavaScript function that returns UI.

Example:

```
function Header() {  
  return <h1>My Website</h1>;  
}
```

This is a component because:

- It is a function.
- Its name starts with a capital letter.
- It returns JSX.
- It can be used like a custom HTML tag.

Use it like this:

```
function App() {  
  return (  
    <main>  
      <Header />  
      <p>Welcome to my app.</p>  
    )  
}
```

```
    </main>  
  );  
}
```

Simple definition:

A component is a reusable piece of UI.

7. Why Components Are Useful

Components help you split a large UI into smaller parts.

Instead of building one large page in one file, you can create small components with clear jobs.

Example page structure:

```
App  
├─ Header  
├─ AboutBox  
├─ SkillBadge  
└─ Footer
```

Each component has one responsibility:

- **Header** shows the top part of the page.
- **AboutBox** shows information about the person or app.
- **SkillBadge** shows a skill label.
- **Footer** shows bottom information.

Components make code:

- Easier to read.
- Easier to reuse.
- Easier to test later.
- Easier to fix.
- Easier to organize.

Beginner rule:

If a part of the UI has a clear job, it may become a component.

8. Component Names Must Start With a Capital Letter

Correct:

```
function ProfileCard() {  
  return <div>Profile</div>;  
}
```

Incorrect:

```
function profileCard() {  
  return <div>Profile</div>;  
}
```

Why?

React treats lowercase JSX tags as normal HTML elements.

These are normal HTML elements:

```
<div></div>  
<p></p>  
<section></section>
```

This is a custom React component:

```
<ProfileCard />
```

React knows `ProfileCard` is a component because it starts with a capital letter.

Beginner rule:

Component names should use PascalCase.

Examples of PascalCase:

```
Header  
ProfileCard
```


SkillBadge
AboutBox
MainLayout

9. Using Components Inside Other Components

A component can use another component.

Example:

```
function Header() {  
  return <h1>My Profile</h1>;  
}  
  
function App() {  
  return (  
    <main>  
      <Header />  
      <p>Welcome to my profile page.</p>  
    </main>  
  );  
}
```

Here, `App` uses `Header`.

This is called component composition.

Simple definition:

Component composition means building a bigger UI by combining smaller components.

10. Example: Before Components

Without splitting into components, your `App.jsx` might look like this:

```
export default function App() {  
  return (  
    <main>  
      <h1>My Profile</h1>  
    </main>  
  );  
}
```

```

    <section>
      <h2>About Me</h2>
      <p>I am learning React.</p>
    </section>

    <section>
      <h2>Skills</h2>
      <p>JavaScript, React, CSS</p>
    </section>
  </main>
);
}

```

This works, but as the page grows, `App.jsx` can become messy.

11. Example: After Splitting Into Components

You can split the same UI like this:

```

function Header() {
  return <h1>My Profile</h1>;
}

function AboutBox() {
  return (
    <section>
      <h2>About Me</h2>
      <p>I am learning React.</p>
    </section>
  );
}

function SkillsBox() {
  return (
    <section>
      <h2>Skills</h2>
      <p>JavaScript, React, CSS</p>
    </section>
  );
}

export default function App() {
  return (
    <main>

```

```
    <Header />
    <AboutBox />
    <SkillsBox />
  </main>
);
}
```

This is cleaner because:

- `Header` handles the heading.
- `AboutBox` handles the about section.
- `SkillsBox` handles the skills section.
- `App` controls the page structure.

12. The One Parent Element Rule

A component must return one parent value.

This is wrong:

```
function App() {
  return (
    <h1>Hello</h1>
    <p>Welcome</p>
  );
}
```

Why is it wrong?

Because React sees two sibling elements being returned directly:

```
h1
p
```

React needs one parent wrapper.

12.1 Fix With a `div`

```
function App() {
  return (
```

```
    <div>
      <h1>Hello</h1>
      <p>Welcome</p>
    </div>
  );
}
```

This works because there is now one parent element: `div`.

12.2 Better Fix With Semantic HTML

Instead of using `div` for everything, use meaningful HTML when possible.

```
function App() {
  return (
    <main>
      <h1>Hello</h1>
      <p>Welcome</p>
    </main>
  );
}
```

`main` is better here because it describes the main content of the page.

Beginner rule:

Use meaningful HTML elements when they match the purpose.

Useful semantic elements:

```
<header>
<main>
<section>
<article>
<nav>
<footer>
```

13. What Are Fragments?

Sometimes you need to return multiple elements, but you do not want to add an extra `div` to the page.

React fragments solve this problem.

Example:

```
function Greeting() {  
  return (  
    <>  
      <h1>Hello</h1>  
      <p>Welcome to React</p>  
    </>  
  );  
}
```

The fragment is this part:

```
<>  
  ...  
</>
```

It groups JSX elements together without adding an extra HTML element to the DOM.

Simple definition:

A fragment is an invisible wrapper for JSX.

13.1 Why Use a Fragment?

Use a fragment when:

- You need to return multiple JSX elements.
- You do not need a real wrapper element like `div`, `main`, or `section`.
- You want to avoid unnecessary extra HTML.

Example with unnecessary `div`:

```
function Greeting() {  
  return (  
    <div>  
      <h1>Hello</h1>  
      <p>Welcome</p>  
    </div>  
  );  
}
```

```
    );  
  }  
}
```

Example with fragment:

```
function Greeting() {  
  return (  
    <>  
      <h1>Hello</h1>  
      <p>Welcome</p>  
    </>  
  );  
}
```

The fragment version does not create an extra `div`.

13.2 Long Fragment Syntax

You can also write fragments like this:

```
import { Fragment } from "react";  
  
function Greeting() {  
  return (  
    <Fragment>  
      <h1>Hello</h1>  
      <p>Welcome</p>  
    </Fragment>  
  );  
}
```

Most beginners use the short version:

```
<>  
  ...  
</>
```

14. Files and Components

In small examples, you can write multiple components in `App.jsx`.

But in real projects, you should split components into files.

Example structure:

```
src/  
  App.jsx  
  components/  
    Header.jsx  
    ProfileCard.jsx  
    SkillBadge.jsx
```

This makes your app easier to maintain.

15. What Is an Export?

An export makes code from one file available to another file.

Example:

```
export default function Header() {  
  return <h1>My React App</h1>;  
}
```

This means:

Make `Header` available so another file can import and use it.

16. What Is an Import?

An import brings code from another file into the current file.

Example:

```
import Header from "../components/Header";
```

This means:

Bring the `Header` component into this file so I can use `<Header />`.

17. Default Exports

A file can have one default export.

Example:

```
// src/components/Header.jsx

export default function Header() {
  return <h1>My React App</h1>;
}
```

Import it like this:

```
// src/App.jsx

import Header from "../components/Header";

export default function App() {
  return (
    <main>
      <Header />
    </main>
  );
}
```

Important detail:

```
import Header from "../components/Header";
```

There are no curly braces.

Why?

Because `Header` is a default export.

Beginner rule:

Default imports do not use `{}`.

17.1 One Default Export Per File

This is allowed:

```
export default function Header() {  
  return <h1>Header</h1>;  
}
```

This is not allowed in the same file:

```
export default function Header() {  
  return <h1>Header</h1>;  
}  
  
export default function Footer() {  
  return <footer>Footer</footer>;  
}
```

A file can only have one default export.

18. Named Exports

A file can have many named exports.

Example:

```
// src/components/ProfileCard.jsx  
  
export function ProfileCard() {  
  return (  
    <section>  
      <h2>Sangeeth</h2>  
      <p>React learner</p>  
    </section>  
  )  
}
```

```
    );  
  }
```

Import it like this:

```
import { ProfileCard } from "../components/ProfileCard";
```

Named exports require curly braces.

Beginner rule:

Named imports use `{ }`.

18.1 Multiple Named Exports in One File

Example:

```
// src/components/Animals.jsx  
  
export function Cat() {  
  return <p>Cat</p>;  
}  
  
export function Dog() {  
  return <p>Dog</p>;  
}  
  
export function Bird() {  
  return <p>Bird</p>;  
}
```

Import them like this:

```
import { Cat, Dog, Bird } from "../components/Animals";
```

Use them like this:

```
function App() {  
  return (  
    <main>  
      <Cat />  
    </main>  
  );  
}
```

```

    <Dog />
    <Bird />
  </main>
);
}

```

19. Default Export vs Named Export

Topic	Default Export	Named Export
How many per file?	One	Many
Import uses <code>{}</code> ?	No	Yes
Import name can be changed easily?	Yes	Usually must match the exported name
Common use	Main component of a file	Multiple utilities or multiple components

Example default export:

```

export default function Header() {
  return <h1>Header</h1>;
}

```

Import:

```

import Header from "./Header";

```

Example named export:

```

export function ProfileCard() {
  return <section>Profile</section>;
}

```

Import:

```

import { ProfileCard } from "./ProfileCard";

```

Simple memory rule:

Default export: no braces. Named export: braces.

20. Common Import Mistake

Suppose `Header.jsx` has this:

```
export default function Header() {  
  return <h1>Header</h1>;  
}
```

Correct import:

```
import Header from "../Header";
```

Incorrect import:

```
import { Header } from "../Header";
```

Why is it wrong?

Because `Header` was exported as default, not as a named export.

21. Another Common Import Mistake

Suppose `ProfileCard.jsx` has this:

```
export function ProfileCard() {  
  return <section>Profile</section>;  
}
```

Correct import:

```
import { ProfileCard } from "../ProfileCard";
```

Incorrect import:

```
import ProfileCard from "../ProfileCard";
```

Why is it wrong?

Because `ProfileCard` was exported as a named export, not a default export.

22. Complete Sprint 2 Example Project

Create this structure:

```
src/  
  App.jsx  
  components/  
    Header.jsx  
    ProfileCard.jsx  
    SkillBadge.jsx
```

22.1 `Header.jsx`

```
export default function Header() {  
  const title = "My React Profile";  
  
  return (  
    <header>  
      <h1>{title}</h1>  
      <p>I am learning components, JSX, and imports.</p>  
    </header>  
  );  
}
```

Concepts shown:

- Default export.
 - Component name starts with a capital letter.
 - JSX expression with `{title}`.
 - Semantic HTML with `header`.
-

22.2 ProfileCard.jsx

```
export function ProfileCard() {
  return (
    <section className="card">
      <h2>Sangeeth</h2>
      <p>Frontend learner building React step by step.</p>
    </section>
  );
}
```

Concepts shown:

- Named export.
 - `className` instead of `class`.
 - Component returning JSX.
 - Semantic HTML with `section`.
-

22.3 SkillBadge.jsx

```
export function SkillBadge() {
  return <span className="skill-badge">React Beginner</span>;
}
```

Concepts shown:

- Named export.
 - Self-contained reusable component.
 - `className` usage.
-

22.4 App.jsx

```
import Header from "../components/Header";
import { ProfileCard } from "../components/ProfileCard";
import { SkillBadge } from "../components/SkillBadge";

export default function App() {
  return (
    <main>
      <Header />
    </main>
  );
}
```

```

    <section>
      <h2>About</h2>
      <ProfileCard />
    </section>

    <section>
      <h2>Current Skill</h2>
      <SkillBadge />
    </section>
  </main>
);
}

```

Concepts shown:

- Default import.
- Named imports.
- Component composition.
- One parent element.
- Semantic `main` and `section` elements.

23. What This Example Demonstrates

This complete example demonstrates:

- JSX syntax.
- JavaScript expressions inside JSX.
- `className`.
- React components.
- Capitalized component names.
- Self-closing custom components.
- Component composition.
- Default exports.
- Named exports.
- Imports from separate files.
- Basic project organization.

24. Beginner Project Structure

For a very small beginner project, this structure is enough:

```
src/  
  main.jsx  
  App.jsx  
  components/  
    Header.jsx  
    ProfileCard.jsx  
    SkillBadge.jsx
```

`main.jsx` starts the app.

`App.jsx` is the main app component.

`components/` stores reusable UI components.

25. Production-Friendly Beginner Structure

As projects grow, do not keep every file directly inside `src/`.

A better structure is:

```
src/  
  components/  
  pages/  
  hooks/  
  services/  
  context/  
  utils/  
  assets/
```

25.1 `components/`

Use `components/` for reusable UI pieces.

Examples:

```
Button.jsx  
Card.jsx  
Navbar.jsx
```



```
ProfileCard.jsx  
SkillBadge.jsx
```

Beginner rule:

If a UI piece can be reused in more than one place, put it in `components/`.

25.2 `pages/`

Use `pages/` for full screens or route-level views.

Examples:

```
Home.jsx  
Products.jsx  
Login.jsx  
Dashboard.jsx
```

Beginner rule:

If it represents a full screen or route, put it in `pages/`.

You will use this more when you learn routing later.

25.3 `hooks/`

Use `hooks/` for custom hooks.

Examples:

```
useFetch.js  
useDebounce.js  
useLocalStorage.js
```

You do not need to master this in Sprint 2. This becomes important later.

25.4 `services/`

Use `services/` for API or backend communication.

Examples:

```
api.js
authService.js
productService.js
```

Beginner rule:

Put browser/API communication in `services/` so components can focus on UI.

25.5 `context/`

Use `context/` for shared app state providers.

Examples:

```
AuthContext.jsx
ThemeContext.jsx
```

You do not need this deeply in Sprint 2. You will study it later.

25.6 `utils/`

Use `utils/` for pure helper functions.

Examples:

```
formatPrice.js
capitalizeName.js
countDone.js
```

Beginner rule:

Put general helper functions in `utils/`, not inside components.

25.7 `assets/`

Use `assets/` for static files.

Examples:

```
logo.png  
hero.jpg  
icons/  
images/
```

26. Sprint 2 Folder Focus

For Sprint 2, mainly focus on:

```
components/  
pages/
```

Do not overcomplicate the project yet.

At this stage, your main goal is:

Learn how to move reusable UI into `components/` and keep `App.jsx` clean.

27. Reusable Components vs Page-Level Components

A reusable component is a small UI piece that can be used in many places.

Examples:

```
Button  
Card  
Navbar  
SkillBadge  
ProfileCard
```

A page-level component is a full screen.

Examples:

HomePage
LoginPage
ProductsPage
DashboardPage

Simple comparison:

Type	Purpose	Folder
Reusable component	Small UI piece	components/
Page component	Full screen or route	pages/

28. Debug Practice

Broken code:

```
function App() {  
  return <h1>Hello</h1>  
  <p>React</p>  
}  
  
function profilecard() {  
  return <div class="card">Card</div>  
}  
  
import { Header } from "../Header";
```

This code has several problems.

28.1 Problem 1: Multiple Elements Without a Parent

Broken:

```
function App() {  
  return <h1>Hello</h1>  
  <p>React</p>  
}
```

`App` is trying to return two elements directly:

```
h1
p
```

Fix with a fragment:

```
function App() {
  return (
    <>
      <h1>Hello</h1>
      <p>React</p>
    </>
  );
}
```

28.2 Problem 2: Component Name Starts With Lowercase

Broken:

```
function profilecard() {
  return <div class="card">Card</div>
}
```

Correct:

```
function ProfileCard() {
  return <div className="card">Card</div>;
}
```

React components should start with a capital letter.

28.3 Problem 3: `class` Should Be `className`

Broken:

```
<div class="card">Card</div>
```

Correct:

```
<div className="card">Card</div>
```

28.4 Problem 4: Import May Be Wrong

Broken:

```
import { Header } from "../Header";
```

This is only correct if `Header` was exported as a named export:

```
export function Header() {  
  return <h1>Header</h1>;  
}
```

If `Header` was exported as default:

```
export default function Header() {  
  return <h1>Header</h1>;  
}
```

Then the import should be:

```
import Header from "../Header";
```

28.5 Corrected Debug Code

```
import Header from "../Header";  
  
function ProfileCard() {  
  return <div className="card">Card</div>;  
}  
  
export default function App() {  
  return (  
    <>  
      <Header />  
      <h1>Hello</h1>  
    </>  
  );  
}
```

```
    <p>React</p>
    <ProfileCard />
  </>
);
}
```

This fixed version uses:

- Correct import syntax for a default export.
- Capitalized component name.
- `className`.
- Fragment wrapper.
- One default exported `App` component.

29. Sprint 2 Practice Task

Build a personal intro page.

The page should use these components:

```
Header
AboutBox
SkillBadge
```

Suggested structure:

```
src/
  App.jsx
  components/
    Header.jsx
    AboutBox.jsx
    SkillBadge.jsx
```

Your page should show:

- A heading.
 - A short intro paragraph.
 - An about section.
 - At least three skill badges.
 - Components imported from separate files.
-

30. Practice Task Example

Header.jsx

```
export default function Header() {  
  return (  
    <header>  
      <h1>My React Profile</h1>  
      <p>I am learning React step by step.</p>  
    </header>  
  );  
}
```

AboutBox.jsx

```
export function AboutBox() {  
  return (  
    <section>  
      <h2>About Me</h2>  
      <p>I know JavaScript and I am now learning React components.</p>  
    </section>  
  );  
}
```

SkillBadge.jsx

```
export function SkillBadge() {  
  return <span className="skill-badge">React</span>;  
}
```

App.jsx

```
import Header from "../components/Header";  
import { AboutBox } from "../components/AboutBox";  
import { SkillBadge } from "../components/SkillBadge";
```



```
export default function App() {
  return (
    <main>
      <Header />
      <AboutBox />

      <section>
        <h2>Skills</h2>
        <SkillBadge />
        <SkillBadge />
        <SkillBadge />
      </section>
    </main>
  );
}
```

This works, but it has one limitation:

All three skill badges show the same text.

That is acceptable for Sprint 2 if you have not studied props yet.

In Sprint 3, you will learn props, which allow this:

```
<SkillBadge label="JavaScript" />
<SkillBadge label="React" />
<SkillBadge label="CSS" />
```

For now, focus on components and imports.

31. Expected Output

Your page should look conceptually like this:

```
My React Profile

I am learning React step by step.

About Me
I know JavaScript and I am now learning React components.

Skills
React
```

React
React

After Sprint 3, you will improve this using props so each badge can show different text.

32. Sprint 2 Challenge Task

Create an `Animals.jsx` file with:

- One default component.
- Two named components.

Example:

```
// src/components/Animals.jsx

export default function AnimalTitle() {
  return <h2>Animals</h2>;
}

export function Cat() {
  return <p>Cat</p>;
}

export function Dog() {
  return <p>Dog</p>;
}
```

Import all three into `App.jsx`:

```
import AnimalTitle, { Cat, Dog } from "../components/Animals";

export default function App() {
  return (
    <main>
      <AnimalTitle />
      <Cat />
      <Dog />
    </main>
  );
}
```

This challenge helps you practice mixing default and named exports from the same file.

33. Important Mental Models

33.1 JSX Mental Model

JSX = UI description written inside JavaScript

React reads JSX and uses it to decide what to show on the screen.

33.2 Component Mental Model

Component = function that returns UI

A React app is a tree of components.

Example:

```
<App />  
├─ <Header />  
├─ <AboutBox />  
└─ <SkillBadge />
```

33.3 Fragment Mental Model

Fragment = invisible wrapper

Use it when JSX needs one parent wrapper but you do not want an extra HTML element.

33.4 Export/Import Mental Model

Export = make code available
Import = bring code into another file

33.5 Project Structure Mental Model

```
components/ = reusable UI pieces
pages/ = full screens
services/ = API communication
utils/ = helper functions
assets/ = images and static files
```

34. Common Beginner Mistakes

Mistake 1: Using `class` Instead of `className`

Wrong:

```
<div class="card">Card</div>
```

Correct:

```
<div className="card">Card</div>
```

Mistake 2: Lowercase Component Names

Wrong:

```
function profileCard() {
  return <div>Profile</div>;
}
```

Correct:

```
function ProfileCard() {
  return <div>Profile</div>;
}
```

Mistake 3: Returning Multiple Elements Without a Parent

Wrong:

```
function App() {  
  return (  
    <h1>Hello</h1>  
    <p>React</p>  
  );  
}
```

Correct:

```
function App() {  
  return (  
    <>  
      <h1>Hello</h1>  
      <p>React</p>  
    </>  
  );  
}
```

Mistake 4: Importing Default Export With Braces

If the file has:

```
export default function Header() {  
  return <h1>Header</h1>;  
}
```

Wrong:

```
import { Header } from "../Header";
```

Correct:

```
import Header from "../Header";
```

Mistake 5: Importing Named Export Without Braces

If the file has:

```
export function ProfileCard() {  
  return <section>Profile</section>;  
}
```

Wrong:

```
import ProfileCard from "../ProfileCard";
```

Correct:

```
import { ProfileCard } from "../ProfileCard";
```

Mistake 6: Forgetting to Close Tags

Wrong:

```

```

Correct:

```

```

Mistake 7: Putting Everything in `App.jsx`

This may be fine at the beginning.

But as your app grows, move reusable pieces into `components/`.

Better:

```
src/  
  App.jsx
```

```
components/  
  Header.jsx  
  ProfileCard.jsx
```

35. Revision Checkpoint Questions

Use these questions to test yourself.

Question 1: What is JSX?

JSX is syntax that lets you write UI code that looks like HTML inside JavaScript.

Question 2: Why does JSX use `className` instead of `class`?

Because `class` is a reserved word in JavaScript. JSX uses `className` for CSS classes.

Question 3: Why must component names usually start with a capital letter?

React treats lowercase tags as normal HTML elements and capitalized tags as custom components.

Question 4: What problem do fragments solve?

Fragments let a component return multiple JSX elements without adding an extra HTML element to the page.

Question 5: How do you import a named export?

Use curly braces.

Example:

```
import { ProfileCard } from './ProfileCard';
```

Question 6: How do you import a default export?

Do not use curly braces.

Example:

```
import Header from './Header';
```

Question 7: Which folder should hold reusable components?

Use the `components/` folder.

Question 8: Which folder should hold route-level pages?

Use the `pages/` folder.

Question 9: What belongs in `services/`?

API calls and backend communication helpers belong in `services/`.

Question 10: What belongs in `utils/`?

Pure helper functions belong in `utils/`.

Examples:

```
formatPrice  
capitalizeName  
countDone
```

36. Sprint 2 Self-Check List

You are ready to move on from Sprint 2 if you can do these things without notes:

- Explain JSX in simple words.

- Write JSX with a variable inside `{}`.
 - Use `className` correctly.
 - Create a function component.
 - Name components with capital letters.
 - Use a component inside another component.
 - Explain the one parent element rule.
 - Use a fragment.
 - Create a component in a separate file.
 - Export a component as default.
 - Export a component as a named export.
 - Import a default export correctly.
 - Import a named export correctly.
 - Explain what belongs in `components/`.
 - Explain what belongs in `pages/`.
 - Build a simple personal intro page using separate component files.
-

37. Sprint 2 Final Summary

Sprint 2 is about learning how React UI is structured.

The most important ideas are:

- JSX lets you write UI inside JavaScript.
- JSX looks like HTML but follows JavaScript rules.
- Use `className`, not `class`.
- Use `{}` to insert JavaScript expressions into JSX.
- Every JSX tag must close.
- A React component is a function that returns UI.
- Component names should start with capital letters.
- Components help split a large UI into smaller reusable pieces.
- A component must return one parent value.
- Fragments let you group JSX without adding extra HTML.
- Default exports are imported without braces.
- Named exports are imported with braces.
- `components/` holds reusable UI pieces.
- `pages/` holds full screens or route-level components.
- A clean project structure helps your app stay maintainable as it grows.

Final beginner sentence:

In Sprint 2, I learn to write JSX, create React components, split them into files, use fragments, and import/export components correctly so my React app stays clean and organized.

38. Mini Revision Sheet

JSX

UI-like syntax written inside JavaScript.

Component

A function that returns UI.

`className`

React JSX version of HTML `class`.

`{}` in JSX

Used to insert JavaScript expressions.

Fragment

An invisible wrapper for multiple JSX elements.

Default Export

One main export from a file.

Import without braces:

```
import Header from './Header';
```

Named Export

One of many possible exports from a file.

Import with braces:

```
import { ProfileCard } from './ProfileCard';
```

`components/`

Reusable UI pieces.

`pages/`

Full screens or route-level views.

39. Practice Prompt

Explain this out loud:

JSX lets me write UI inside JavaScript. A React component is a function that returns JSX. Component names should start with capital letters. If I return multiple elements, I need one parent element or a fragment. I use default exports when a file has one main component, and named exports when I want to export several things. Reusable UI goes inside components, and full screens go inside pages.

If you can explain that clearly and build the practice page, Sprint 2 is complete.